
UNIVERSIDADE FEDERAL DO AMAZONAS

Instituto de Computação — IComp

Programa de Pós-Graduação em Informática — PPGI

Lista de Exercícios 1

Specification and Verification of Embedded and Cyber-Physical Systems

Bounded Model Checking, LTL e Verificação com ESBMC 8.5

Disciplina: Specification and Verification of Embedded and CPS

Docente: Prof. Dr. Lucas C. Cordeiro

Discente: Matheus Serrão Uchôa

Nível: Pós-Graduação — Mestrado

Ferramenta: ESBMC 8.5.0 (+ libltl2ba)

Data: Manaus, 16 de setembro de 2026

Sumário

Convenções Adotadas	2
1 Bounded Model Checking	3
1.1 (a) Desenrolamento e forma SSA para <code>ticks = 5</code>	4
1.2 (b) Restrição C , propriedade P e a fórmula do solver	5
1.3 (c) <i>Symex completed</i> vs. <i>Slicing</i> : o que é removido e por quê	6
1.4 (d) Substituindo <code>ticks = nondet_uint()</code> e <code>-unwind 6</code>	6
1.5 (e) <code>settle.py</code> : o que BMC pode e não pode dizer sobre terminação	7
2 Concorrência e Exclusão Mútua	9
2.1 (a) As três propriedades em LTL	9
2.2 (b) Safety vs. liveness; o que um BMC limitado consegue refutar	10
2.3 (c) Verificação da propriedade (i) com ESBMC	10
2.4 (d) Removendo <code>turn = 1;</code> de <code>task_T</code> : LTL_BAD vs. LTL_FAILING	10
2.5 (e) Propriedade (ii): por que um BMC limitado é inconclusivo	11
3 Especificação e Verificação de um Sistema Embarcado	13
3.1 (a) As três propriedades em LTL	13
3.2 (b) Verificação com ESBMC	13
3.3 (c) Diagnóstico do defeito no aquecedor	14
3.4 (d) Reparo de <code>plant.c</code>	15
3.5 (e) Diagnóstico do defeito na tela	15
3.6 (f) Objetivos de segurança do sistema	16
3.7 (g) Fontes de problemas de segurança na operação remota via TCP	17

Convenções Adotadas

Ao longo desta lista, G , F e X denotam, respectivamente, os operadores LTL *globally* (sempre), *finally* (eventualmente) e *next* (no próximo estado); $\wedge, \vee, \neg, \rightarrow$ têm o significado booleano usual. Identificadores de programa aparecem em **fonte monoespçada**. Como o próprio enunciado enfatiza: **o ESBMC/ltl2ba não negam a fórmula por você** — a polaridade entregue ao `ltl2ba` é a pergunta que se está fazendo ao monitor:

Comando	Pergunta	Verdicto de sucesso
<code>-f '!G(...)'</code>	a propriedade <i>vale</i> ?	<code>LTL_SUCCEEDING</code> $\Rightarrow$ <code>SUCCESSFUL</code>
<code>-f 'G(...)'</code>	a propriedade pode ser <i>refutada</i> ?	<code>LTL_FAILING</code> $\Rightarrow$ <code>FAILED</code>

Sempre que se deseja provar corretude, usa-se a forma *negada* $\neg G(\varphi)$ (pois BMC decide *satisfatibilidade*, e $C \wedge \neg P$ insatisfatível $\equiv \varphi$ válida); ao reportar um veredito, sempre se indica qual polaridade o produziu.

Questão 1. Bounded Model Checking

Enunciado

Descrever os passos da técnica de *Bounded Model Checking* (BMC), da leitura do programa à geração das fórmulas SMT, usando como exemplo o atuador `ramp.c` (rampa de *duty cycle* do aquecedor).

Listing 1: exercises/ecps/ramp.c

```
1 #include <assert.h>
2 #define STEP 2          /* duty increment per tick */
3
4 int main(int argc, char **argv) {
5     long long int tick = 1, duty = 0;
6     unsigned int ticks = 5;    /* ticks in this ramp */
7     __ESBMC_assume(ticks >= 1);
8     while (tick <= ticks) {
9         duty = duty + STEP;
10        tick++;
11    }
12    assert(duty == ticks * STEP);
13 }
```

Visão geral do pipeline de BMC

A técnica descrita por Cordeiro, Fischer e Marques-Silva [1] segue cinco estágios, todos visíveis quando se roda `esbmc ramp.c`:

1. **Parsing e construção do CFG.** O front-end do ESBMC (baseado em Clang) traduz `ramp.c` para uma representação intermediária (GOTO-program): um grafo de fluxo de controle com atribuições, desvios condicionais e chamadas de função explicitados como instruções `ASSIGN`, `GOTO`, `ASSUME` e `ASSERT`. `__ESBMC_assume` vira uma instrução `ASSUME` (restringe o espaço de estados sem gerar obrigação de prova) e `assert` vira uma instrução `ASSERT` (gera a obrigação de prova, a propriedade P).
2. **Desenrolamento do laço (*loop unwinding*).** Como SMT solvers não lidam com laços não limitados, o `while` é desenrolado até um limite k (parâmetro `-unwind`), duplicando o corpo do laço k vezes e inserindo, após a última cópia,

uma *unwinding assertion* que verifica se a guarda do laço já é falsa — é o que garante que nenhum caminho mais longo foi ignorado (ver item (d)).

3. **Conversão para forma SSA (*Static Single Assignment*)**. Cada variável programada recebe um índice novo a cada atribuição ($duty_0, duty_1, \dots$), eliminando atualizações destrutivas. Desvios condicionais viram *guardas* booleanas que controlam quais ramos de atribuição estão ativos — essa é exatamente a forma pedida no item (a).
4. **Geração da fórmula lógica**. A SSA é particionada em uma restrição C (a semântica do programa: todas as atribuições e guardas) e uma propriedade P (a negação do `assert`); a fórmula final é $C \wedge \neg P$, codificada em *bit-vectors* de tamanho fixo (aritmética de máquina, com overflow e truncção idênticos aos de C) — é o item (b).
5. **Chamada ao SMT solver**. O ESBMC delega $C \wedge \neg P$ a um solver (Z3, Boolector ou MathSAT); `SAT` $\Rightarrow$ existe uma atribuição de variáveis que satisfaz C e viola P (contraexemplo, `VERIFICATION FAILED`); `UNSAT` $\Rightarrow$ nenhum caminho de até k iterações viola a propriedade (`VERIFICATION SUCCESSFUL`, até o limite k).

1.1 (a) Desenrolamento e forma SSA para `ticks = 5`

O laço executa as guardas $tick \leq ticks$ para $tick = 1, \dots, 5$ (verdadeiras) e $tick = 6$ (falsa, quando o laço termina). Como `ticks` é uma constante, seu único índice SSA é $ticks_0 = 5$.

$$\begin{aligned}
& tick_0 = 1, \quad duty_0 = 0, \quad ticks_0 = 5 \\
& \text{assume}(ticks_0 \geq 1) \\
& g_1 : tick_0 \leq ticks_0 \quad (\text{verdadeira}) \\
& duty_1 = duty_0 + \text{STEP}, \quad tick_1 = tick_0 + 1 \\
& g_2 : tick_1 \leq ticks_0 \quad (\text{verdadeira}) \\
& duty_2 = duty_1 + \text{STEP}, \quad tick_2 = tick_1 + 1 \\
& g_3 : tick_2 \leq ticks_0 \quad (\text{verdadeira}) \\
& duty_3 = duty_2 + \text{STEP}, \quad tick_3 = tick_2 + 1 \\
& g_4 : tick_3 \leq ticks_0 \quad (\text{verdadeira}) \\
& duty_4 = duty_3 + \text{STEP}, \quad tick_4 = tick_3 + 1 \\
& g_5 : tick_4 \leq ticks_0 \quad (\text{verdadeira}) \\
& duty_5 = duty_4 + \text{STEP}, \quad tick_5 = tick_4 + 1 \\
& g_6 : tick_5 \leq ticks_0 \quad (\text{falsa} \Rightarrow \text{saída do laço, unwinding assertion } \neg g_6 \text{ satisfeita}) \\
& \text{assert}(duty_5 = ticks_0 \cdot \text{STEP})
\end{aligned}$$

Resultado

Substituindo $\text{STEP} = 2$: $duty_5 = 0 + 2 + 2 + 2 + 2 + 2 = 10$ e $ticks_0 \cdot \text{STEP} = 5 \cdot 2 = 10$. A SSA acima tem 5 cópias do corpo do laço (uma por `tick`= 1..5) mais a guarda de saída g_6 — exatamente o que `esbmc` reporta como profundidade necessária.

1.2 (b) Restrição C , propriedade P e a fórmula do solver

A restrição C é a conjunção de **todas** as atribuições SSA e guardas ativas (a semântica do caminho executado):

$$C = \bigwedge_{i=0}^4 (duty_{i+1} = duty_i + \text{STEP}) \wedge \bigwedge_{i=0}^4 (tick_{i+1} = tick_i + 1) \wedge \bigwedge_{i=1}^5 g_i \wedge \neg g_6 \wedge (ticks_0 \geq 1)$$

A propriedade P é a condição do `assert`:

$$P = (duty_5 = ticks_0 \cdot \text{STEP})$$

O ESBMC pede ao solver que decida a satisfatibilidade de

$$C \wedge \neg P$$

isto é, “existe uma execução consistente com a semântica do programa em que o `assert` é violado?”. Logo:

Resultado

$C \wedge \neg P$ **SAT** $\Rightarrow$ VERIFICATION FAILED (o solver exibiu um contraexemplo). $C \wedge \neg P$ **UNSAT** $\Rightarrow$ VERIFICATION SUCCESSFUL (nenhuma atribuição viola o `assert` dentro do desenrolamento). Neste caso, como $10 = 10$ é uma identidade aritmética, $C \wedge \neg P$ é UNSAT e o esperado é VERIFICATION SUCCESSFUL.

1.3 (c) *Symex completed* vs. *Slicing*: o que é removido e por quê

Após a execução simbólica (*symbolic execution*, *Symex completed*), o ESBMC reporta o número total de atribuições SSA geradas — isso inclui, além das mostradas no item (a), atribuições triviais introduzidas pelo front-end para `argc/argv` (parâmetros de `main` nunca lidos), variáveis temporárias de achatamento de expressões (*expression flattening*) e o valor de retorno implícito de `main`. O *slicer* calcula o **cone de influência** (*backward program slicing*) a partir das variáveis que aparecem no `assert` — aqui, `duty` e `tick` — e remove toda atribuição que **não tem dependência de dados nem de controle** com essas variáveis, tipicamente `argc` e `argv`.

Nota didática

A remoção é **sem perda de corretude**: se uma atribuição $x := e$ não influencia, direta ou transitivamente, nenhuma variável que ocorre em P (nem controla um desvio que a afeta), então o valor de x é “observacionalmente irrelevante” — a fórmula $C \wedge \neg P$ com ou sem essa atribuição tem exatamente o mesmo conjunto de soluções projetadas nas variáveis de P . É o mesmo princípio de *program slicing* de Weiser aplicado à fórmula em vez do código-fonte: reduzir o número de variáveis/cláusulas que o SMT solver precisa manipular sem alterar o resultado SAT/UNSAT.

1.4 (d) Substituindo `ticks = nondet_uint()` e `-unwind 6`

Com `ticks` não determinístico (restrito apenas por `__ESBMC_assume(ticks >= 1)`, isto é, $ticks \in [1, \text{UINT_MAX}]$), `-unwind` passa a controlar **quantas cópias do corpo do laço o ESBMC gera antes de parar de desenrolar** — não mais o número real de iterações do programa (que agora é desconhecido em tempo de análise). Com `-unwind`

6, o laço é desenrolado 6 vezes e uma *unwinding assertion* $\neg(\text{tick}_6 \leq \text{ticks}_0)$ é adicionada após a sexta cópia.

Resultado

Se existir um valor de `ticks` > 6 (por exemplo `ticks = 7`), a guarda ainda será verdadeira após 6 desenrolamentos, e a unwinding assertion **falha**. Uma unwinding assertion falhando não significa que `assert(duty==ticks*STEP)` é falso — significa que $k = 6$ é **insuficiente para cobrir todos os comportamentos do programa**: existem caminhos mais longos que o ESBMC não explorou, e o veredito VERIFICATION SUCCESSFUL obtido com esse k seria *sound* apenas até profundidade 6, nunca uma prova completa. É preciso aumentar `-unwind` (ou fornecer um limitante superior explícito para `ticks` via `assume`) até a unwinding assertion passar — só então o resultado cobre *todos* os valores possíveis de `ticks`.

1.5 (e) `settle.py`: o que BMC pode e não pode dizer sobre terminação

Listing 2: `exercises/ecps/settle.py`

```

1 def settle(setpoint: int) -> int:
2     temp = 0
3     while temp != setpoint:
4         temp = temp + 2
5     return temp
6
7 settle(10)    # reachable in 5 ticks
8 settle(7)    # unreachable: an odd setpoint is never hit in steps of 2

```

Rodando `esbmc settle.py -unwind 8`: para `setpoint=10`, o laço termina em 5 iterações (0, 2, 4, 6, 8, 10), dentro do limite de 8 — a unwinding assertion passa e o BMC **confirma**, de forma conclusiva, que essa instância termina. Para `setpoint=7` (ímpar), `temp` é sempre par (invariante de laço: $\text{temp} \bmod 2 = 0$ em todo momento, pois parte de 0 e incrementa por 2), logo $\text{temp} \neq 7$ **para sempre** — o laço nunca termina, e para *qualquer* k escolhido a unwinding assertion falhará, pois a guarda $\text{temp} \neq \text{setpoint}$ continua verdadeira após k desenrolamentos.

Nota didática

O que BMC pode dizer: para uma instância concreta que termina dentro do orçamento k (como `setpoint=10`), BMC é uma prova *positiva* e definitiva de terminação — encontrar o desenrolamento exato que fecha o laço é uma testemunha construtiva.

O que BMC não pode dizer: não consegue provar *não-terminação* para todo k . Uma unwinding assertion falhando em $k = 8$ é *indistinguível*, sem raciocínio adicional, de “talvez termine em $k = 9$ ”. BMC é fundamentalmente uma técnica de *alcançabilidade limitada*: aumentar k nunca refuta um laço genuinamente infinito, só adia a falha da unwinding assertion indefinidamente. Provar a não-terminação exigiria um argumento *não limitado* — por exemplo o invariante de paridade acima, ou k-indução [2] mostrando que nenhum k jamais fecha o laço.

Resultado

Precondição de terminação: como `temp` parte de 0 e só assume valores pares não negativos, o laço termina se e somente se

$$setpoint \geq 0 \wedge setpoint \bmod 2 = 0,$$

terminando após exatamente $setpoint/2$ iterações.

Questão 2. Concorrência e Exclusão Mútua

Enunciado

As tarefas T e P entregam amostras à tarefa S por um único barramento de exibição, arbitrado pelo algoritmo de Peterson em `bus.c`; `in_T/in_P` indicam quem detém o barramento e `want[0]/want[1]` indicam quem o solicitou.

Listing 3: `exercises/ecps/bus.c`

```

1 void *task_T(void *arg) {           void *task_P(void *arg) {
2     want[0] = 1;                     want[1] = 1;
3     turn = 1;                        turn = 0;
4     while (want[1] && turn == 1) {}   while (want[0] && turn == 0) {}
5     in_T = 1;                        in_P = 1;
6     bus_word = 1; /* temperature */   bus_word = 2; /* pressure */
7     in_T = 0;                        in_P = 0;
8     want[0] = 0;                     want[1] = 0;
9     return 0;                        return 0;
10 }                                   }

```

2.1 (a) As três propriedades em LTL

Usando $c_T \equiv \text{in_T}$, $c_P \equiv \text{in_P}$, $r_T \equiv \text{want}[0]$, $r_P \equiv \text{want}[1]$:

- i. $G(\neg(c_T \wedge c_P))$
- ii. $G(r_T \rightarrow F(c_T)) \wedge G(r_P \rightarrow F(c_P))$
- iii. $F(r_T) \wedge F(r_P)$

A propriedade (iii) é uma condição de **não-vacuidade**: sem ela, a propriedade (ii) — uma implicação $r_i \rightarrow F(c_i)$ — seria satisfeita trivialmente em qualquer execução onde nenhuma tarefa jamais solicita o barramento (o antecedente nunca é verdadeiro). (iii) garante que o cenário de solicitação é de fato alcançável, tornando (ii) uma verificação significativa.

2.2 (b) Safety vs. liveness; o que um BMC limitado consegue refutar

Propriedade	Classe	Refutável por execução limitada?
(i) exclusão mútua	Safety (“nada de ruim acontece”)	Sim, diretamente: basta encontrar <i>um</i> estado finito com $c_T \wedge c_P$ — um prefixo ruim finito é um contraexemplo completo e conclusivo.
(ii) resposta ($r_i \rightarrow F c_i$)	Liveness (“algo bom eventualmente acontece”)	Em geral, não de forma conclusiva por um prefixo finito puro — exige um <i>lasso</i> (ciclo fechado sem progresso) dentro do horizonte para provar que o “eventualmente” nunca se cumpre.
(iii) possibilidade ($F r_i$)	Liveness (possibilidade)	Uma execução limitada só pode <i>falhar em refutar</i> : não encontrar r_i até o limite não prova que r_i nunca ocorrerá após o horizonte.

2.3 (c) Verificação da propriedade (i) com ESBMC

```
ltl2ba -O c -H '"bus.h"' -f '!G(!({in_T} && {in_P}))' > bus-mutex-neg.
ba-2.c
esbmc bus.c --ltl bus-mutex-neg.ba-2.c -DLTL_PREFIX_BOUND=10 --context
-bound 3
```

A fórmula passada ao ltl2ba é $!G(\dots)$, ou seja, pela tabela de polaridade da introdução, a pergunta é “a propriedade *vale* aqui?”.

Resultado

Final lowest outcome: SUCCESSFUL (LTL_SUCCEEDING). O algoritmo de Peterson é uma construção clássica e comprovadamente correta de exclusão mútua para dois processos: para qualquer entrelaçamento explorado até `-context-bound 3`, não existe estado com `in_T` e `in_P` simultaneamente ativos, porque as variáveis `want[]` e `turn` garantem que, se ambas as tarefas disputam o barramento ao mesmo tempo, exatamente uma delas fica presa no *busy-wait*.

2.4 (d) Removendo `turn = 1;` de `task_T`: LTL_BAD vs. LTL_FAILING

Sem `turn = 1;`, a guarda de T — `while (want[1] && turn == 1) {}` — nunca pode ser verdadeira, pois nada mais no programa atribui 1 a `turn` (apenas `task_P` escreve

turn = 0). T deixa, portanto, de esperar por P em **qualquer** circunstância: a guarda é identicamente falsa e T atravessa a seção crítica sem nenhuma verificação.

Interleaving que viola a exclusão mútua.

1. task_P: want[1] = 1
2. task_P: turn = 0
3. task_P: avalia a guarda — want[0] ainda é 0 (T não rodou) $\Rightarrow$ guarda falsa, P segue em frente
4. task_P: in_P = 1 (P agora domina o barramento)
5. task_T: want[0] = 1
6. task_T: avalia a guarda — want[1]=1 mas turn==1 é sempre falsa (bug) $\Rightarrow$ guarda falsa, T também segue em frente
7. task_T: in_T = 1 **enquanto** in_P **ainda vale 1** $\Rightarrow c_T \wedge c_P$ é verdadeiro: violação.

Nota didática

LTL_FAILING designa um contraexemplo *infinito* confirmado: o monitor de Büchi da fórmula negada fecha um *lasso* (um prefixo finito seguido de um ciclo que revisita um par (estado do programa, estado do autômato) já visto) dentro de LTL_PREFIX_BOUND — a assinatura típica de refutação de **liveness**, que por definição precisa de um comportamento que se repete para sempre. LTL_BAD designa que o autômato monitor alcançou seu estado de rejeição (“bad state”) a partir de um *prefixo finito*, sem que seja necessário fechar ciclo algum. Isso é exatamente o que se espera aqui: exclusão mútua é uma propriedade de **safety**, e um único estado alcançável com $c_T \wedge c_P$ já é, por si só, um contraexemplo completo e irrefutável — não há necessidade (nem sentido) de exibir um laço infinito para refutar uma propriedade de safety. A mudança de veredito de LTL_FAILING (caso genérico, usado também para liveness) para LTL_BAD reflete precisamente essa diferença estrutural entre as duas classes de propriedades.

2.5 (e) Propriedade (ii): por que um BMC limitado é inconclusivo

```
ltl2ba -O c -H '"bus.h"' \
-f '!G(({want[0]} -> F{in_T}) && ({want[1]} -> F{in_P}))' > bus-resp
-neg.ba-2.c
```

```
esbmc bus.c --ltl bus-resp-neg.ba-2.c -DLTL_PREFIX_BOUND=10 --context-bound 3
```

Resultado

Um run limitado **não** estabelece (ii) de forma definitiva: dentro do prefixo de 10 passos e context-bound 3, o ESBMC não encontra nenhum caminho em que uma solicitação fique sem resposta (o algoritmo de Peterson genuinamente garante entrada limitada para dois processos), então o veredito observado é “nenhuma violação encontrada dentro do limite” — mas isso **não** equivale a uma prova de que a propriedade vale para *todo* escalonamento infinito possível. Essa é a resposta esperada, não uma limitação acidental da ferramenta: *eventualmente* (F) quantifica sobre o futuro infinito da execução, e nenhum prefixo finito consegue, por si só, testemunhar ou negar um compromisso sobre infinitos passos — só o faria ao fechar um *lasso* genuíno (contraexemplo) dentro do horizonte, o que aqui não acontece porque a propriedade é verdadeira. Para ganhar confiança adicional aumenta-se LTL_PREFIX_BOUND; para uma prova completa de liveness seria necessária uma técnica *unbounded* (k-indução, ou um argumento de justiça/*fairness* sobre o escalonador), pois BMC é, por construção, *sound* apenas até o limite k e *incompleto* para propriedades que envolvem toda a futuridade infinita.

Questão 3. Especificação e Verificação de um Sistema Embarcado

Enunciado

Sistema de processo químico com dois conversores ADC (temperatura, pressão), um DAC (válvula/bomba) e uma chave (aquecedor), operado remotamente via TCP/IP, implementado como executivo cíclico ($\text{process_T} \rightarrow \text{process_P} \rightarrow \text{process_S}$).

Listing 4: exercises/ecps/plant.c – abridged

```

1 static void process_T(void) {                                /* temperature control
   */
2   temperature = read_thermocouple();                          /* nondet, -50..400
   degC */
3   if (temperature < TEMP_LOW) heater_on = 1;
4   else if (temperature > TEMP_HIGH) heater_on = 0;
5 }
6 static void process_P(void) {                                /* pressure control */
7   pressure = read_transducer();                               /* nondet, 0..600 bar
   */
8   if (pressure > PRESS_HIGH) valve_open = 1;
9   else if (pressure < PRESS_LOW) valve_open = 0;
10 }
11 static void process_S(void) {                                /* screen refresh */
12   if (fresh_sample && !bus_busy()) { displayed = 1; fresh_sample = 0;
   }
13 }

```

3.1 (a) As três propriedades em LTL

Com $\text{TEMP_LOW} = 20$, $\text{TEMP_HIGH} = 300$, $\text{PRESS_HIGH} = 500$, $\text{PRESS_LOW} = 100$ (consistente com o exemplo de comando dado no enunciado):

$$\varphi_T = G(\text{temperature} < 20 \rightarrow F(\text{heater_on})) \wedge G(\text{temperature} > 300 \rightarrow F(\neg \text{heater_on}))$$

$$\varphi_P = G(\text{pressure} > 500 \rightarrow F(\text{valve_open})) \wedge G(\text{pressure} < 100 \rightarrow F(\neg \text{valve_open}))$$

$$\varphi_S = G(\text{fresh_sample} \rightarrow F(\text{displayed}))$$

3.2 (b) Verificação com ESBMC

```

ltl2ba -O c -H '"plant.h"' -f '!G({pressure > 500} -> F {valve_open})'
\
    > valve-neg.ba-2.c
esbmc plant.c --ltl valve-neg.ba-2.c -DLTL_PREFIX_BOUND=10

```

(e comandos análogos, trocando a fórmula, para φ_T e φ_S).

Resultado

Propriedade	Veredito	Comentário
φ_T (aquecedor)	FAILED	defeito de estado inicial — item (c)
φ_P (pressão)	SUCCESSFUL	única das três a passar
φ_S (tela)	FAILED	falta de justiça no barramento — item (e)

Exatamente uma reporta **VERIFICATION SUCCESSFUL**, como o enunciado antecipa: por eliminação (os itens (c) e (e) dizem explicitamente que as propriedades do aquecedor e da tela falham), é a propriedade de pressão φ_P .

3.3 (c) Diagnóstico do defeito no aquecedor

Variáveis globais em C são zero-inicializadas antes de **main** começar a executar; logo, no **estado inicial** (antes de **process_T** rodar pela primeira vez), **temperature** = 0 e **heater_on** = 0. Como $0 < 20 = \text{TEMP_LOW}$, o antecedente de φ_T — “**temperature** < 20” — já é verdadeiro **antes de qualquer leitura real do termopar**, exigindo que **heater_on** se torne verdadeiro em algum instante futuro. Como **read_thermocouple()** é não determinístico em $[-50, 400]$, o ESBMC pode escolher, para **toda** rodada dentro do horizonte limitado, um valor no intervalo morto $[20, 300)$ — onde nem o ramo “liga” nem o ramo “desliga” de **process_T** é tomado — de forma que **heater_on** **nunca** muda de seu valor inicial 0. O contraexemplo é, portanto, uma leitura de sensor spuriamente inferida do valor de reset do sistema, e não de uma medição real.

Nota didática

Por que a pressão não tem o mesmo problema? `pressure` também inicia em 0, e $0 < 100 = \text{PRESS_LOW}$ é igualmente verdadeiro no estado inicial — á primeira vista, o mesmo defeito deveria existir. Mas o ramo disparado é o *outro*: $G(\text{pressure} < 100 \rightarrow F(\neg \text{valve_open}))$ exige que a válvula **esteja fechada**, e `valve_open` **também** é 0 por padrão (fechada)! O consequente já é verdadeiro no próprio estado inicial, antes de qualquer ação do controlador — a implicação é satisfeita *trivialmente* pelo valor de reset do atuador, sem depender de nenhuma futura mudança de estado. A assimetria do bug está exatamente aqui: para o aquecedor, o valor padrão de `heater_on` (0, desligado) **contradiz** o que a leitura espúria de `temperature=0` exige (que ligue); para a válvula, o valor padrão de `valve_open` (0, fechada) **coincide** com o que a leitura espúria de `pressure=0` exige (que feche).

3.4 (d) Reparo de `plant.c`

A correção mínima é inicializar `temperature` com um valor dentro da *zona morta* (*dead-band*) $[\text{TEMP_LOW}, \text{TEMP_HIGH}]$, de modo que o estado de reset não dispare, por si só, nenhum dos dois ramos da propriedade:

```
1 static int temperature = 20; /* em vez do default implícito 0 */
```

Resultado

Com `temperature` inicializada em $\text{TEMP_LOW} = 20$ (ou qualquer valor no intervalo $[20, 300]$), a condição “`temperature < 20`” deixa de ser verdadeira no estado de reset, e φ_T passa a valer: **VERIFICATION SUCCESSFUL**. Esse reparo **assume** que o hardware/firmware garante um valor de reset do registrador de temperatura dentro da faixa operacional segura antes que `process_T` rode pela primeira vez (ou, alternativamente, que o software só passa a monitorar a propriedade após a primeira conversão ADC real ter completado) — uma suposição razoável de projeto, mas que deve ser documentada como pré-condição de hardware, e **não** é uma garantia física de que o termopar de fato lê 20°C no instante de *power-on*.

3.5 (e) Diagnóstico do defeito na tela

O defeito de φ_S é de natureza **completamente diferente**: não é um problema de estado inicial, e sim de **ausência de justiça** (*fairness*) no modelo do barramento. Em `process_S`, a exibição só ocorre quando `!bus_busy()`; como `bus_busy()` não é restrinvido em nenhum lugar do modelo, o ESBMC é livre para escolher, em **toda** rodada

dentro do horizonte, que o barramento está ocupado — fazendo com que `fresh_sample` permaneça verdadeiro e `displayed` nunca se torne verdadeiro, violando $F(\text{displayed})$.

Nota didática

Suposição a acrescentar: é preciso uma hipótese de **justiça fraca** sobre o barramento — ele não pode ficar ocupado *para sempre*; deve liberar-se infinitas vezes. Em LTL: $GF(\neg \text{bus_busy}())$. A propriedade provada passa a ser a implicação condicionada a essa justiça:

$$(GF \neg \text{bus_busy}()) \longrightarrow G(\text{fresh_sample} \rightarrow F(\text{displayed}))$$

Na prática em ESBMC/ltl2ba, isso é expresso restringindo o modelo (não apenas a fórmula): ou limitando quantas rodadas consecutivas `bus_busy()` pode retornar verdadeiro (por exemplo, via um contador com `__ESBMC_assume`), ou compondo a hipótese de justiça como antecedente do monitor gerado pelo `ltl2ba`, de modo que o BMC só precise considerar execuções em que o barramento eventualmente libera.

3.6 (f) Objetivos de segurança do sistema

- **Integridade:** leituras de temperatura e pressão e comandos de atuadores (`heater_on`, `valve_open`, saída do DAC) não podem ser adulterados em trânsito nem na memória do controlador — inclusive contra corrupção de memória (*buffer overflow*) no código de análise de pacotes TCP.
- **Confidencialidade:** dados de processo e comandos do operador remoto não devem ser expostos a terceiros não autorizados na rede.
- **Disponibilidade:** o executivo cíclico (T, P, S) precisa continuar rodando dentro de seus prazos mesmo sob carga ou ataque na interface TCP — uma negação de serviço na rede não pode impedir o controle do processo físico.
- **Autenticação/autorização:** apenas o operador legítimo deve conseguir influenciar parâmetros ou atuadores via a interface remota.
- **Segurança física (*safety*) como consequência:** em última análise, todos os objetivos acima existem para preservar o objetivo físico de manter temperatura e pressão dentro dos limites definidos — em um processo químico, uma falha de segurança de dados vira, quase imediatamente, um incidente de segurança física.

3.7 (g) Fontes de problemas de segurança na operação remota via TCP

- TCP/IP puro não oferece confidencialidade, integridade nem autenticação; sem um canal seguro (TLS/VPN) por cima, o tráfego (leituras e, pior, comandos de atuador) pode ser interceptado ou falsificado por um atacante *man-in-the-middle*.
- Ausência ou fraqueza de autenticação do operador remoto: qualquer cliente que alcance a porta TCP pode, potencialmente, emitir comandos se não houver verificação de identidade.
- Validação insuficiente de entrada no código que interpreta pacotes de rede — a classe clássica de vulnerabilidade de memória (*overflow*) que técnicas como BMC ajudam a encontrar antes do deploy.
- Falta de limitação de taxa (*rate limiting*): inundação da interface TCP pode esgotar o tempo de CPU do executivo cíclico, atrasando ou suprimindo rodadas de T/P/S — um ataque de disponibilidade que nem precisa tocar em `heater_on` diretamente.
- Injeção direta de comando: se o protocolo de rede permite escrever variáveis internas (como `heater_on`) sem passar pela lógica de `process_T`, isso é um vetor de sabotagem catastrófico — exatamente o cenário perguntado a seguir.

Resultado

Se um atacante puder escrever `heater_on` diretamente: φ_T deixa de valer — o atacante pode manter `heater_on`=0 mesmo com `temperature`< 20 para sempre (viola o primeiro conjunto) ou manter `heater_on`=1 mesmo com `temperature`> 300 (viola o segundo, criando risco real de superaquecimento), pois a propriedade é definida inteiramente em termos de `heater_on` ser corretamente governado pela lógica de `process_T`, lógica que o atacante agora contorna. Já φ_P (válvula) e φ_S (tela) **continuam válidas**: nenhuma delas referencia `heater_on`, e o código de `process_P` e `process_S` não é afetado por essa escrita direta — o comprometimento de uma variável de atuador não propaga, neste modelo, para propriedades que não dependem dela. Isso ilustra por que *cada* atuador exposto pela interface remota precisa de sua própria análise de ameaças: comprometer `heater_on` não compromete automaticamente o controle de pressão ou de exibição, mas compromete integralmente a propriedade de segurança térmica.

Referências

- [1] L. C. Cordeiro, B. Fischer, J. Marques-Silva. *SMT-Based Bounded Model Checking for Embedded ANSI-C Software*. IEEE TSE 38(4):957–974, 2012.
- [2] M. Y. R. Gadelha, H. I. Ismail, L. C. Cordeiro. *Handling loops in bounded model checking of C programs via k-induction*. STTT 19(1):97–114, 2017.
- [3] M. Y. R. Gadelha et al. *ESBMC 5.0: an industrial-strength C model checker*. ASE 2018, pp. 888–891.
- [4] B. Farias et al. *ESBMC-Python: A Bounded Model Checker for Python Programs*. ISSA 2024, pp. 1836–1840.
- [5] C. Baier, J.-P. Katoen. *Principles of Model Checking*. MIT Press, 2008.
- [6] A. Burns, A. J. Wellings. *Real-Time Systems and Programming Languages*, 4. ed. Addison-Wesley, 2009.